14.3.3 Defining a configuration overlay for staging

In the previous sections, we used Kustomize to manage the configuration of Catalog Service in a local development environment. Those manifests will represent the *base* for multiple customizations applied for each environment as *overlays*. Since we'll define overlays in the `polar-deployment` repository while the base is in the `catalog-service` repository, all the Catalog Service manifests must be available in the main remote branch. If you haven't done so yet, push all the changes applied to your Catalog Service project so far to the remote repository on GitHub.

NOTE As I explained in chapter 2, I expect you have created a different repository on GitHub for each project in the Polar Bookshop system. In this chapter we're working only with the `polar-deployment` and `catalog-service` repositories, but you should have also created repositories for `edge-service`, `order-service`, and `dispatcher-service`.

As anticipated, we'll store any configuration overlay in the `polar-deployment` repository. In this section and the following ones, we'll define an overlay for the staging environment. The next chapter will cover production.

Go ahead and create a new kubernetes/applications folder in your `polar-deployment` repository. We'll use it to keep the customizations for all the applications in the Polar Bookshop system. In the newly created path, add a catalog-service folder that will contain any overlay for customizing the deployment of Catalog Service in different environments. In particular, we'll want to prepare the deployment in staging, so create a "staging" folder for Catalog Service.

Any customization (base or overlay) requires a kustomization.yml file. Let's create one for the staging overlay of Catalog Service (polar-